A Hierarchical Taxonomy for Tracking AI-Assisted Mathematical Progress

Standalone specification for evidence coding and a future taxonomy explorer

Mathematics Progress project

Version 0.2.0 — 4 August 2026

Status. This is the versioned v0.2 operational specification. It defines the stable hierarchy used for browsing, two coding profiles, a separately versioned challenge-context layer and the richer metadata retained on individual evidence records. It is intended to evolve through explicit additions, deprecations, splits and mappings rather than silent relabelling. The v0.2 definitions must be tested through an inter-coder pilot before they are treated as mature.

Abstract

This document specifies a hierarchical, faceted taxonomy for tracking mathematical results produced with artificial-intelligence, computational and formal methods. It is designed for two uses: consistent coding in a systematic evidence review and progressive exploration in a public interface. The taxonomy has three top-level browse facets: *Mathematics*; *Result and mathematical approach*; and *Production and verification*. Each facet supports drill-down from a short, stable label to evidence-backed detail. A linked *challenge-context* layer records problem history and derives the project's Expected, Difficult and Superhuman categories without becoming a fourth taxonomy facet. Minimal and Full coding profiles separate scalable inclusion from deeper analysis. Exact model versions, repositories, prompts, run settings and verification events are deliberately retained as record metadata rather than promoted into the main navigation tree. The classification unit is the mathematical result, linked to one or more papers and artifacts. This prevents a computer-science venue label from obscuring the mathematical field of a result, and prevents the subject codes of a multi-result paper from being copied indiscriminately to every result.

Contents

1 Purpose and scope

The taxonomy answers three plain-language questions:

1. **Where in mathematics does the result belong?**
2. **What kind of result was obtained, and what broad mathematical approach produced it?**
3. **How was it produced, and how was it checked?**

It is not intended to classify all mathematics exhaustively from first principles. The mathematics branch reuses the Mathematics Subject Classification (MSC2020). The two project-specific branches describe the form of the contribution, the production pipeline and the verification evidence. The taxonomy supports results ranging from informal AI-generated proofs through computer-assisted constructions to kernel-checked formal proofs.

The intended corpus is an evolving systematic record rather than a static benchmark. The full-text, manually validated and temporally aware review design used by Cruz et al. (2025) motivates four requirements here: classifications should be based on substantive source evidence rather than titles alone; labels should have recorded provenance; changes should be versioned; and aggregate trends should remain reproducible when the corpus or vocabulary changes.

2 Design commitments and precedents

2.1 A faceted hierarchy, not one giant tree

Mathematical subject, contribution type and verification are orthogonal. A single tree would force implausible paths such as placing a kernel-checked graph-theory counterexample underneath a particular model name. The project therefore uses three independent hierarchical facets. A result receives one primary path in each applicable browse subtree and may receive secondary paths where the evidence warrants them.

Problem history and the project's research-challenge category are linked analytical metadata rather than a fourth facet. They answer a different question—how the target relates to a documented prior frontier—and must not alter the subject, result or production classification. Likewise, coding completeness is recorded by profile and must not be mistaken for confidence, verification or mathematical importance.

2.2 Reuse established schemes where they fit

MSC2020 is maintained jointly by Mathematical Reviews and zbMATH and supplies the canonical subject hierarchy (Mathematical Reviews and zbMATH, 2020). Its linked-data representation demonstrates how broader concepts, preferred labels, cross-references and version mappings can be expressed using SKOS (Arndt et al., 2021; Miles and Bechhofer, 2009). Model Cards and PaperCard supply precedent for recording model identity, version, intended use, access date, assistance stages and limitations as metadata (Mitchell et al., 2019; Cho et al., 2023). The autonomous-agent literature supplies stable architectural distinctions such as planning, memory, action and tool use (Wang et al., 2024). CRediT provides a useful precedent for multi-role contribution statements, while remaining a taxonomy of human rather than machine contribution (National Information Standards Organization, 2022).

Formal verification is represented as a sequence of verification events. Lean's own guidance makes clear that kernel acceptance concerns a precise formal statement relative to definitions, imports and axioms; it does not by itself establish that the statement faithfully represents the intended informal claim (Lean FRO, 2026). Artifact status is likewise separated from result validity, following the distinction between repeatability, reproducibility and replicability used in artifact-review practice (Association for Computing Machinery, 2020).

2.3 Ambiguity is recorded, not hidden

Mathematical classification is not perfectly deterministic. Schubotz et al. (2020) report only about 81% agreement between zbMATH and Mathematical Reviews even at coarse primary-field level. The schema therefore records primary and secondary paths, assignment provenance, a short rationale and confidence. The interface should not imply more precision than the evidence supports.

3 Entity model: results, papers and artifacts

The **classification unit is a result record**. A result is a bounded mathematical claim package: for example, a theorem, counterexample, construction, exact computation, improved bound or formalisation target. It may be supported by several papers and artifacts. Conversely, one paper may contain several separately classified results.

Entity	Purpose	Typical contents
Result	Unit used for mathematical coverage and progress counts.	Canonical claim, outcome, subject path, result/approach path, production/verification paths, status and coding evidence.
Paper	Bibliographic and publication context.	Authors, title, date, venue, arXiv categories, author-supplied MSC codes, peer-review status and links to the results it reports.
Artifact	Reproducible object used to produce or check a result.	Formal project, proof script, code, prompts, transcripts, data, certificate, repository commit, environment and licence.
System	Reusable model or compound method.	Model family and snapshot, harness, agent roles, retrieval sources, tools, access mode and configuration.
Verification event	One check applied to one claim or artifact.	Method, verifier, independence, scope, outcome, date, assumptions and evidence URI.

Entity	Purpose	Typical contents
Problem-history record	Versioned context for a named target or frontier.	Anchor claim, framing source/date, prior status, attempts, best known result, aliases, disputes and links to result records.
Challenge assessment	Derived analytical record linked to one result and problem-history record.	P/A/S/R component values, evidence, missingness, metric version, total and category.

This separation resolves two recurring problems. First, an AI-systems paper filed under computer science may contain a result whose mathematical target belongs in number theory or algebra. Second, a multi-result mathematics paper may list several MSC codes that should not all be assigned to every theorem in the paper.

4 Facet I: Mathematics

4.1 Canonical hierarchy

The primary mathematical path uses MSC2020 and is assigned according to the *mathematical target of the tracked result*, not the paper's venue or the technology used to obtain it.

Level	Source	Meaning	Example
M1	MSC two-digit code	Broad mathematical field used for aggregate coverage.	`13` Commutative algebra
M2	MSC three-character code	Subfield within the field.	`13J` Topological rings and modules
M3	MSC five-character code	Most specific canonical MSC topic.	`13J10` Complete rings, completion
M4	Local named concept	Named problem, conjecture or result family, versioned by this project.	Anderson quasi-completeness problem

Every result has exactly one **primary** MSC path when a defensible code exists. It may have zero or more **secondary** MSC paths for an essential setting, application or cross-domain contribution. Secondary paths are not included in primary coverage counts unless an analysis explicitly opts into them.

4.2 Assignment fields

- `classification_basis`: normally `mathematical_target`; alternatives must be explained.
- `scheme` and `scheme_version`: normally `MSC/MSC2020`.
- `primary_path`: ordered M1–M3 nodes, each with code and preferred label.
- `secondary_paths`: zero or more ordered MSC paths, each with a relation and rationale.

- `named_problem`: optional M4 project concept with a stable identifier, aliases, provenance and links to its separately maintained problem-history record.
- `external_classifications`: author, arXiv, zbMATH and MathSciNet labels preserved verbatim.
- `assignment_status`: `confirmed`, `probable` or `provisional`.
- `confidence`: `high`, `medium` or `low`, supported by a short rationale.

Technology-related MSC codes such as theorem proving or formalisation may be appropriate for a systems paper. They do not replace the production taxonomy and should not be added to the mathematical result merely because Lean or another prover was used.

5 Facet II: Result and mathematical approach

This facet lets a reader browse first by what was achieved and then, if desired, by the broad argument form and specialist technique.

5.1 Hierarchy

Level	Name	Question answered
R1	Contribution family	Is this a mathematical result, formalisation, system/tool, benchmark/evaluation, or survey/synthesis?
R2	Outcome	Was a target resolved affirmatively, resolved negatively, partly advanced, improved, classified or computed?
R3	Argument form	Was the result obtained by proof, counterexample, construction, reduction, finite search, certificate or another broad form?
R4	Specialist technique	Which mathematically meaningful technique distinguishes the argument? This is an extensible controlled-local vocabulary.

5.2 R1–R2 controlled vocabulary

Choose one primary contribution path for display and counting. Add a secondary contribution path only when the paper genuinely makes two independently useful contributions, such as a new mathematical result and a new formalisation system.

R1 family	R2 child	Use when
Mathematical result	Affirmative resolution	A stated target or implication is proved in its intended scope.
Mathematical result	Negative resolution	A conjecture, universal assertion or proposed implication is disproved.

R1 family	**R2 child**	**Use when**
Mathematical result	Undecidability or impossibility	A decision, approximation or construction task is shown impossible under stated assumptions.
Mathematical result	Partial progress	A meaningful case, direction or conditional result is established without settling the target.
Mathematical result	Improved bound	A previous numerical, asymptotic or extremal bound is strengthened but not shown sharp.
Mathematical result	Improved algorithm	A complexity, approximation or resource guarantee is improved without a matching optimality result.
Mathematical result	Construction or witness	The main contribution is an explicit or existential mathematical object.
Mathematical result	Exact value or classification	A quantity is determined exactly, or all objects in the stated class are characterised.
Formalisation	New formalisation	An existing or newly produced mathematical result is encoded and checked in a proof assistant.
Formalisation	Formal library or infrastructure	Reusable definitions, lemmas or interfaces are the primary contribution.
System or tool	Discovery or proving system	A reusable AI, search or formalisation system is introduced or materially extended.
Benchmark or evaluation	Capability evaluation	Systems are compared on a defined set of mathematical tasks.
Survey or synthesis	Evidence synthesis	Prior work is organised, reviewed or quantitatively mapped without a new target theorem.

5.3 R3 argument forms

R3 is a set-valued classification because a proof can combine reduction, construction and induction without one form subsuming the others. A coder may nominate one `display_primary` value when the source makes an organising form clear and the explorer needs a short label. This nomination is optional, has no analytical priority and never removes the other R3 values. If no form is defensibly primary, the explorer displays a neutral multi-form summary.

R3 label	Definition
Direct proof	Establishes the target without making negation of the conclusion the organising strategy.
Proof by contradiction	Uses an assumed negation of the target as the principal global strategy.
Counterexample	Refutes a universal assertion by providing an object satisfying the assumptions and violating the conclusion.
Constructed existence	Establishes existence through a construction, including long recursive or prescribed-object constructions.
Nonconstructive existence	Establishes existence without producing a usable witness or construction procedure.
Reduction or characterisation	Converts the target to an equivalent or sufficient condition that can be resolved.
Induction or recursion	Uses induction, structural recursion, transfinite recursion or a related iterative construction as a principal method.
Probabilistic existence	Uses random choice or a probabilistic estimate to establish existence.
Exhaustive finite search	Proves a finite claim by complete enumeration of the relevant search space.
Certificate-backed computation	Produces a result whose correctness is checked by a separate deterministic verifier or certificate checker.
Formal proof synthesis	Search directly produces a proof term or script accepted by a formal prover.
Experimental evidence only	Reports computational or empirical support that does not establish the mathematical claim as a theorem.

5.4 R4 specialist-technique registry

R4 is extensible but controlled. Examples include *discharging*, *flag algebras*, *prescribed completion*, *generic formal-fibre control*, *polynomial method*, *SAT encoding*, *linear programming dual certificate* and *transfinite recursion*. A term must describe a mathematically substantive method, not merely repeat the subject, argument form or name a software product.

Every R4 entry is stored in a registry with a stable identifier, preferred label, definition, aliases, scope note, status, introduction version and at least one evidence-backed usage example. A new term enters as `provisional`; one documented use, a non-circular definition and reviewer approval are sufficient to represent a genuinely novel technique. It becomes `stable` only after a second independent use or an explicit vocabulary review confirms that it is reusable. Synonyms map to one identifier. Rejected and deprecated terms remain in the change log so that old records can be migrated. Coders may propose terms but must not mint arbitrary labels directly in result records.

6 Facet III: Production and verification

The third facet has two sibling subtrees: **Production** and **Verification**. They appear together in the interface because readers often ask how a result was produced and how much assurance supports it, but they must remain distinct in the data.

6.1 Production hierarchy

Level	Name	Purpose
P1	Human–machine mode	Records who led the mathematically substantive stages.
P2	Reasoning representation	Distinguishes informal language, executable computation, formal proof and hybrid pipelines.
P3	Architecture family	Distinguishes direct calls, agentic workflows, program/evolution search, neural-symbolic systems and compound pipelines.
P4	Architecture features	Adds retrieval, generator–verifier loops, planning, memory, parallelism and human approval points.

P1: human–machine mode

Label	Operational definition
Human-led, AI-assisted	Humans own the principal strategy or proof; AI supplies bounded assistance such as search, examples, algebra, checking or editing.
Shared human–AI	Humans and systems make interdependent substantive contributions at the decisive mathematical stage, with neither cleanly described as merely assisting.
AI-led	The system supplies the decisive search, strategy, proof or construction; humans may select, prompt, curate, edit or validate without materially supplying that decisive step.
Source-claimed autonomous	A source explicitly reports no substantive human intervention during the named decisive mathematical stage, supported by a quotation or precise source location. This is not treated as an audited project-wide fact.
Human-only control	A comparison or verification artifact is produced without AI involvement. Included for baselines and later independent checks.
Unclear or not reported	The available sources do not support a defensible allocation.

P1 is supplemented by an event-level allocation table. For each stage—target selection, statement interpretation, retrieval, conjecture generation, strategy, proof construction, computation, formalisation, repair, validation and exposition—the record stores human and machine operations separately. An overall label must never erase this stage-specific account. For a single-valued summary, source-claimed autonomous takes precedence over AI-led only when its stricter stage-and-source rule is satisfied; otherwise classify the allocation at the decisive mathematical stage and use unclear or not reported when the source pack cannot distinguish the cases.

P2: reasoning representation

- **Informal natural-language reasoning**
- **Executable computation or program search**
- **Formal theorem proving**
- **Hybrid informal-to-formal**
- **Hybrid computation-to-formal-certificate**
- **Mixed or not reported**

P3–P4: architecture

Architecture label	Definition
Direct model interaction	One or more bounded prompts without an autonomous iterative controller.
Iterative single-agent	One agent plans or revises across multiple steps using feedback or tools.
Multi-agent	Distinct agents or contexts have specialised roles such as planning, generation, criticism or formal proving.
Program or evolutionary search	Candidate programs or mathematical objects are generated, evaluated and selected over repeated generations.
Neural-symbolic	A learned component is coupled to a symbolic deduction, constraint or proof engine.
Compound pipeline	Distinct informal, computational or formal systems operate in a defined sequence.
Retrieval-augmented	The workflow retrieves mathematical statements, literature, library lemmas or examples during the run.
Generator–critic	A critic gives qualitative feedback to a generator.
Generator–verifier	A verifier applies a defined acceptance check and returns repair feedback or a pass/fail decision.
Planning and task decomposition	A controller constructs or revises subgoals, modules or proof obligations.
Persistent memory	State survives context compression, sessions or independent agent calls.
Parallel candidate search	Multiple candidate proofs, programs or obligations are explored concurrently.

Architecture label	Definition
Human-in-the-loop	Human approval, selection, hints or corrections occur at recorded intervention points.

Exact systems and versions—for example a model snapshot, Rethlas, Lean, Isabelle, a CAS or a SAT solver—are metadata, not stable hierarchy nodes. They may be searchable filters.

6.2 Verification hierarchy

Verification is a multi-valued event tree. Each event records its target, method, verifier, independence, scope and outcome.

Family	Method	What it establishes
Internal	Model self-check	A model or agent judges its own output; useful feedback but minimal independence.
Internal	Different-model check	Another model checks the output; independence depends on harness, data and team.
Human	Expert mathematical review	A qualified person checks some stated scope of the mathematics. Record identity, procedure and independence where known.
Computational	Deterministic tests	Code passes specified tests; establishes only the tested behaviour.
Computational	Numerical or CAS check	Symbolic or numerical calculations support stated identities or cases.
Computational	Exhaustive finite search	All cases in a defined finite space are checked.
Certificate	Separately checked certificate	A small or independently checkable artifact establishes the result relative to checker correctness.
Formal	Proof-assistant kernel check	The formal theorem is accepted relative to its definitions, imports and allowed axioms.
Formal	Statement-correspondence review	A human or comparison tool checks that the formal statement represents the intended informal claim.
External	Independent replay	A different party reruns supplied artifacts in a supplied or equivalent environment.

Family	Method	What it establishes
External	Independent replication	A different party establishes the claim using independently created artifacts or a new proof/formalisation.
Publication	Peer review or publication	The work has passed a venue's process; this is recorded separately from mathematical and formal checks.

Statement correspondence is recorded independently of kernel acceptance. Each formal check therefore links the informal anchor claim to the exact formal declaration and assigns one correspondence value: *equivalent scope*, *stronger formal statement*, *weaker formal statement*, *partial formalisation*, *different target*, or *not assessed*. The record stores the corresponding machine value and states which assumptions, cases, bounds or definitions differ. A kernel-checked proof of a weaker or partial statement remains a valid formal verification event, but it must not be displayed as verification of the broader anchor claim.

For display, the explorer may derive a short assurance summary such as *reported only*, *expert checked*, *mechanically checked*, *formally verified* or *independently reproduced*. Derivation is conservative and versioned: a summary may assert only what a completed, in-scope event establishes; failed, partial and unknown-scope events do not promote it; *formally verified* additionally requires a successful kernel event and an equivalent-scope or stronger-formal-statement assessment. Where events support incomparable descriptions, the explorer uses a compound or neutral summary rather than selecting the most impressive phrase. The summary records the rule version and contributing event identifiers. It is a lossy convenience label, not a total ordering, and all underlying events remain visible and exportable.

7 Detailed metadata outside the browse hierarchy

The following fields are retained on the record detail page and made searchable where useful. They are not promoted into the main tree because they are volatile, highly granular or non-hierarchical.

Metadata group	Fields
Bibliographic	Paper identifier, title, authors, version, date, venue, DOI/arXiv ID, URLs, publication status and external classifications.
Model provenance	Provider, family, exact snapshot if known, access date, access mode, adaptation or fine-tuning, licence and reported limitations.
Harness provenance	Name, version or commit, orchestrator, prompts or skills, planning, memory, retrieval sources, candidate count, selection rule, stopping rule, token/compute/time budget and logs.

Metadata group	Fields
Tools	Proof assistant, library and commit; CAS; programming language; solver; database; search engine; compiler; hardware where material.
Human contribution	Stage-specific specification, prompting, steering, selection, correction, proof, validation, software and writing roles.
Artifacts	Informal proof, formal project, code, certificate, data, prompts, transcript, repository URI, commit/hash, licence and archival location.
Formal trust	Main declaration, statement hash, imports, axioms, admitted gaps, unsafe features, kernel/checker, statement review and replay instructions.
Reproducibility	Availability, build instructions, dependency lock, run settings, repeated runs, independent replay and independent replication.
Problem history	Stable problem identifier, anchor claim, aliases, framing source/date and precision, prior status, prior best result or bound, dated attempts and outcomes, disputed status, solution date and provenance.
Challenge context	Metric version; P/A/S/R values and evidence state; component rationales and sources; derived total/category; provisional flag; confidence; override evidence and assessment history.
Coding provenance	Taxonomy version, coder, assignment date, evidence excerpt/URI, confidence, rationale and review history.

Unknown values are stored explicitly as `unknown` or `not_reported`. They must not be inferred from a system's default configuration unless the record clearly marks the inference and its evidence.

8 Linked challenge-context layer

Challenge context describes the documented relation between a result and a prior mathematical frontier. It is first-class, evidence-backed metadata linked to the result and, where applicable, to an M4 problem-history record. It is **not** a fourth browse facet, an importance score, a verification score or an autonomy score. The challenge metric has its own `challenge_metric_version` (currently 1.0.0) so that its rules can change without changing taxonomy concept identifiers.

8.1 Metric and component evidence

When all four components are known, the derived score is

$$D = P + A + S + R, \qquad 0 \leq D \leq 5.$$

Component	Values	Operational rule
P: prior exact target	0 or 1	Assign 1 when a dated source predating the AI work documents the same question, conjecture, record or quantitative frontier as the anchor claim. Assign 0 only after recording the search scope and finding no exact prior target in the reviewed evidence; otherwise use `unknown`.
A: advance	0, 1 or 2	Assign 0 for a narrow or unquantified extension; 1 for a material improvement, new theorem, construction, counterexample or substantial subcase; and 2 for complete proof/disproof, classification, matching optimum, sharp endpoint or removal of the central condition. The comparison is against the anchor claim, not the paper's rhetoric or publication status.
S: scope	0 or 1	Assign 1 for a uniform or worst-case result over an unbounded or genuinely general class. Otherwise assign 0 when scope is clear, or `unknown` when it is not.
R: demonstrated resistance	0 or 1	Assign 1 for at least two independent pre-solution attempts at the exact target, or for a predeclared target-specific comparison with at least five independent runs or systems and no more than 20% success. Attempts, comparison protocol and sources must be recorded.

Expected is derived for complete scores of 0–2 and Difficult for complete scores of 3–5. **Superhuman** overrides the base label when P=1, A=2, the result status is non-provisional and at least 1.0 decade elapsed from precise problem framing to solution. Duration is derived from recorded dates, with their precision retained. Verification, AI autonomy, prestige, proof length and compute remain separate and add no points.

The term *Superhuman* is deliberate project framing for a narrowly operational research-challenge category. It emphasises that an AI-involved result completely settled a precisely documented target that had remained open to human mathematical work for at least a decade. It does not, by itself, prove that humans could not solve the problem, that the system acted autonomously, or that AI is generally superior to human mathematicians.

Superhuman requires an auditable evidence bundle: a dated pre-solution source stating the anchor target; a dated solution source; an explicit comparison showing that the result settles that same target completely; non-provisional result status; date-precision fields; and second-coder review. The result authors' characterisation alone cannot supply both the prior-status and complete-settlement judgments. Any unresolved mismatch, material mathematical objection or missing required field blocks the override and is displayed, rather than being converted to zero.

8.2 Problem history and missingness

The optional M4 problem-history record is the reusable home for the anchor claim, aliases, framing source and date, prior best result or bound, dated attempts, changes in formulation, known simultaneous discoveries and disputed status. A result record links to it and stores only the assessment-specific snapshot and source identifiers. This allows later timelines without making time a hierarchy level.

Each component stores `value`, `evidence_state`, `rationale`, one or more source references, confidence, coder and assessment date. Permitted evidence states are `observed`, `inferred`, `not_reported`, `unknown` and `not_applicable`. Unknown or not-reported values are never imputed as zero. No total or Expected/Difficult/Superhuman category is derived while a required component is unknown. The interface instead displays *challenge context incomplete* and identifies the missing evidence.

9 Minimal and Full coding profiles

The profile describes record completeness, not evidential quality. A high-confidence Minimal record may be more reliable than a poorly evidenced Full record. Both profiles validate against the same concept registries and preserve explicit missingness.

Area	**Minimal profile**	**Full profile enrichment**
Identity and sources	Stable result ID, canonical anchor claim, at least one dated source and declared source role.	Claim versions, paper/artifact/system links, source excerpts, repositories, archival and licence data.
Mathematics	One primary M1–M3 path, or a coded unclassified reason; assignment status, rationale and confidence.	Secondary MSC paths, M4 problem identity, external classifications and complete problem history.
Result and approach	One R1–R2 path; all evident R3 values or `not_reported`; optional display-primary.	R4 registry identifiers, detailed technique evidence, statement and outcome comparisons.
Production	P1 and P2, each with source, evidence state and confidence.	Stage-specific human/machine events, P3/P4, exact systems, harness, tools, resources and intervention points.
Verification	At least one event, or an explicit `none_reported`; target, family, method, outcome, scope and source.	Verifier identity and independence, assumptions, dates, artifacts, replay/replication, formal trust and statement correspondence.
Challenge context	P/A/S/R component records, each with a value or explicit missingness; metric version and derivation status.	Full problem history, prior attempts, source-independence assessment, date precision, disputes and adjudication history.
Provenance	Taxonomy/profile versions, coder, assignment date, rationale and overall confidence.	Field-level review history, adjudication, migrations and record revision log.

A publisher may release a Minimal record while Full coding is pending. The record exposes `coding_profile`, `profile_version`, validation status and missing Full fields; it must not fabricate values to obtain a completeness badge. Profile requirements may change only through a versioned schema release.

10 Coding protocol

10.1 Step-by-step procedure

1. **Choose the coding profile.** Start with Minimal unless the review question or available evidence requires Full; record the profile version.
2. **Define the result.** Write one canonical anchor claim that is sufficiently precise to distinguish it from other results in the same paper.
3. **Link sources.** Identify papers, announcements, repositories and verification artifacts; assign each a source role.
4. **Assign the mathematical target.** Select one primary MSC path using the actual claim. Preserve author and database codes separately.
5. **Assign the result path.** Choose R1–R2, record every evidenced R3 form and nominate a display-primary only when defensible. Use registry identifiers for R4.
6. **Reconstruct production by stage.** Record human and machine operations before assigning the P1 summary label.
7. **Record architecture.** Add stable P2–P4 labels and exact systems as metadata.
8. **Record verification events.** Do not compress several checks into the word "verified"; compare formal and informal statement scope where applicable.
9. **Code challenge context.** Link or create the problem-history record, code P/A/S/R with sources and retain missingness before deriving any label.
10. **Assess evidence and confidence.** Cite the source supporting each non-obvious assignment and flag source-reported autonomy or correctness claims.
11. **Review required judgments.** A second coder reviews Superhuman overrides and adjudicates uncertain MSC, outcome, autonomy, statement-correspondence and verification assignments.
12. **Freeze the record version.** Store the taxonomy version and date so historical analyses can be reproduced.

10.2 Counting and analysis rules

- Count results, papers and artifacts separately; never mix their denominators.
- Use primary mathematical paths for default concentration and coverage charts.
- Report secondary-path analyses explicitly and expect double counting.
- Aggregate a child into every ancestor in its own path; a result at M3 therefore contributes to M2 and M1 counts.
- Treat corpus coverage as coverage of *observed results*, not a percentage of all mathematics.

- Do not use verification strength, autonomy, prestige, compute or proof length as a proxy for mathematical importance.
- Treat R3 as a set in analysis. The optional display-primary is not a weighting or counting instruction.
- Recompute challenge totals and assurance summaries from their versioned inputs; never store an unexplained manual category as authoritative.
- Distinguish source claims from independent observations in both prose and data.

10.3 Inter-coder pilot and release gate

Before the v0.2 core fields are applied across the corpus, two coders independently code the same stratified sample of 20–30 results. The sample should vary by mathematical field, R2 outcome, production mode, verification family, publication status and evidence completeness. Coders work from the same source pack without discussing assignments, record coding time and flag unclear rules.

Field-level agreement is reported, rather than compressed into one number. Use exact agreement and an appropriate chance-corrected statistic for single-valued primary MSC, R1–R2, P1 and P/A/S/R; use set overlap such as Jaccard similarity for multi-valued R3 and verification families. Missingness agreement is reported separately. Every disagreement is adjudicated with a written reason and classified as coder error, ambiguous evidence, missing decision rule or inadequate vocabulary.

The pilot is a release gate: recurrent ambiguity changes the definition or example set before corpus migration. As diagnostic triggers rather than claims of validity, below 80% exact agreement (or below 0.67 chance-corrected agreement) on a mandatory single-valued field, or mean R3 set overlap below 0.70, requires revision and a focused second pilot. The published pilot report includes the sample, source pack, field statistics, adjudications, coding-time distribution and resulting specification changes.

Pre-release calibration status (4 August 2026)

Two independent AI-assisted coding passes have been compared on 24 stratified records as a decision-rule stress test. This is *not* a human inter-coder reliability study: the source pack contains summaries and links rather than complete page-level excerpts, and six records overlap the worked examples, so blinding is partial. The human release gate therefore remains pending.

Exact agreement was 87.5% for broad MSC, 95.8% for R1, 79.2% for R2, 50.0% for P1, 83.3% for P, 83.3% for A, 91.7% for S and 66.7% for R. R3 exact-set agreement was 50.0% (mean Jaccard 0.722); verification-method exact-set agreement was 66.7% (mean Jaccard 0.750). These are diagnostic observations, not estimates of human reliability. They prioritise sharper P1 precedence, clearer R2 boundary examples, explicit materiality tests for R3, stronger resistance-source requirements and stricter separation of verification method, scope and statement correspondence. The complete machine-readable disagreements are retained with the pilot artifacts.

11 Coding manual: exemplar set and decision guides

The public coding manual should include at least the following seven fully coded, source-cited examples. Together they teach boundaries that a single showcase result cannot reveal.

No.	Example type	Boundary it must demonstrate
1	Complete settlement of a documented long-standing target	Distinguish the anchor claim from a broad topic; show P/A/date evidence and the Superhuman override review.
2	Improved bound or substantial subcase	Distinguish Partial progress and Improved bound from complete settlement; show why A=1 rather than A=2.
3	Counterexample built by construction and reduction	Record several R3 values without forcing an artificial primary; add only genuinely specialist R4 techniques.
4	Exhaustive search or certificate-backed computation	Separate executable production, finite-search argument form, certificate verification and independent replay.
5	New formalisation with equivalent informal scope	Separate mathematical novelty from formalisation novelty; record kernel, axioms and statement-correspondence events.
6	Partial or weaker formalisation	Retain the successful kernel event while preventing a derived summary from overstating the checked target.
7	Systems or benchmark paper containing zero or several result records	Classify the paper contribution separately; split bounded mathematical claims into result records and do not copy venue codes to them.

The manual also supplies short decision flows for six recurrent judgments:

1. **Primary mathematics:** classify the anchor claim, test the most specific defensible MSC code, then record essential secondary settings separately.
2. **Settlement versus progress:** compare the conclusion line by line with the prior anchor target; any unresolved case, altered quantifier or central added assumption blocks complete settlement.
3. **R3 and R4:** record every broad form needed to understand the argument; use R4 only when a registry definition adds mathematical information beyond R3 and MSC.
4. **Human–machine mode:** reconstruct stage events first, then derive P1; never infer autonomy from product branding or absence of an author narrative.
5. **Formal assurance:** record kernel and statement-correspondence events separately; propagate assurance only to the intersection of their scopes.
6. **Challenge context:** establish the precise anchor, prior source and comparison before entering values; use unknown rather than zero when the search or evidence is incomplete.

The Anderson example in this specification demonstrates record-level faceting. The remaining exemplar records, their source excerpts and completed Minimal/Full JSON should be published with the coding manual and updated when an adjudication changes a rule.

12 Explorer presentation and interaction

The explorer should implement progressive disclosure. A collapsed evidence row shows three summaries:

1. **Mathematics**: normally M1 and a short M3 label.
2. **Result and approach**: normally R2 and primary R3.
3. **Production and verification**: normally P1/P2 and the clearest assurance summary.

Challenge context may appear beside these summaries as a clearly separated derived badge; it must not be presented as a fourth taxonomy branch. Incomplete component evidence produces an incomplete badge, not a favourable default category.

Each facet summary expands independently. The first expansion reveals the full primary breadcrumb. A second level reveals secondary paths, techniques, architecture features and verification events. The challenge badge expands to P/A/S/R values, evidence, metric version and rationale. Exact model versions, repository commits, prompts, coding profile and evidence belong in a record detail panel or final expansion.

12.1 Explorer requirements

- Parent filters include all descendants by default.
- Primary and secondary classifications are visually distinct.
- Counts state whether they use primary paths only.
- Codes and human-readable labels are both searchable.
- Deprecated concepts resolve to their replacement but remain available for historical records.
- Keyboard, touch and screen-reader users can expand the same content.
- A record displays "taxonomy not yet assigned" rather than silently disappearing.
- Minimal and Full completeness are labelled separately from confidence and assurance.
- Every assurance summary expands to the contributing event identifiers and derivation-rule version.
- Every challenge category expands to its components, evidence sources and metric version.
- Evidence links and coding confidence are available without leaving the record context.

The hierarchy should remain shallow in the default interface. The number of database fields may grow without making the initial browsing experience more complicated.

13 Colour inheritance in the explorer

Colour is a secondary structural cue, not a substitute for concept identifiers, displayed labels or the visible path. The machine-readable colour map has its own `colour_map_version` (currently 1.0.0). Within a single facet or named classification axis, a curated base hue denotes a parent branch or dimension and its descendants or values retain that hue. Moving from D1 to D4 increases saturation and reduces lightness, so colour can be read as "the same local branch, at greater depth". Darker colour means *more specific*; it does not mean stronger evidence, greater autonomy, higher quality or greater mathematical importance.

The scope of a hue is local. Reusing blue, green or another hue in a different independent facet does not create a taxonomic relationship between those labels. Root containers and top-level facet headings remain neutral, and labels, codes, headings, position and connectors remain authoritative. The interactive explorer activates **one structural colour axis at a time**; values on other axes render neutrally until selected. The appendix displays all registered swatches together only as a reference map.

D1 parent ⟶ D2 child ⟶ D3 grandchild ⟶ D4 deepest   unknown

13.1 Visual grammar

- A **solid connector** denotes parent–child inheritance. A **dashed connector** denotes an independent, combinable axis or a metadata relationship. A **dashed container** denotes an extensible vocabulary.
- Unknown, mixed or not-reported values use neutral grey and a dashed outline. They never receive an automatically selected family colour.
- Stable labels have explicit registry assignments. Hash-based, rotating or otherwise arbitrary fallback colours are not used; an unregistered value is neutral and generates a validation warning.
- Workflow state—for example provisional, deprecated, failed or incomplete—uses a separate status palette and, where useful, an icon or line treatment. Status colours never inherit branch hues and never recolour the taxonomy label itself.
- A visible legend names the active axis and maps every displayed hue to its branch. Switching axes changes the legend and all structural colours together.
- Colour is always redundant with visible text and structure, so the explorer remains usable without colour perception.

13.2 Application by facet

Area	Inheritance rule
Mathematics	Each of the ten display macro-families supplies a hue. An MSC path retains that hue from M1 through M2 and M3 to an optional M4 named concept, while saturation and lightness encode depth. The macro-families are interface groupings, not additional MSC nodes.
Result and approach	Each R1 contribution family supplies a hue that its R2 outcomes inherit. R3 argument form and R4 specialist technique are separate, combinable axes with their own local hues; they are not descendants of a particular R2 outcome.
Production	P1 human–machine mode, P2 reasoning representation, P3 architecture family and P4 architecture features are four sibling dimensions. Each dimension supplies one hue and its values inherit that hue at D2. Their layout must not imply a P1–P2–P3–P4 parent–child chain.
Verification	A verification family is D1 and its methods are D2 descendants in the same hue. The hues are categorical and must never be interpreted as a verification-strength scale.
System metadata	A model or tool family supplies the hue; exact systems and versions inherit it at deeper levels. These labels remain searchable metadata outside the stable browse hierarchy.

The complete structure and colour application appear in Appendix C.

14 Reference data model

The following illustrative JSON shape separates the three browse facets from detailed metadata. It is normative in meaning, not in property naming.

```
{
  "taxonomyVersion": "0.2.0",
  "codingProfile": {"name": "Minimal", "version": "1.0.0"},
  "colourMapVersion": "1.0.0",
  "recordType": "result",
  "resultId": "result-003",
  "anchorClaim": "...",
  "sources": [
    {"paperId": "...", "role": "mathematical exposition"},
    {"artifactId": "...", "role": "formal verification"}
  ],
  "facets": {
    "mathematics": {
      "primaryPath": [
        {"code": "13", "label": "Commutative algebra"},
        {"code": "13J", "label": "Topological rings and modules"},
        {"code": "13J10", "label": "Complete rings, completion"}
```

```
      ],
      "secondaryPaths": [],
      "namedProblemId": "problem-anderson-quasi-completeness"
    },
    "resultApproach": {
      "primaryPath": ["Mathematical result", "Negative resolution"],
      "argumentForms": ["Counterexample", "Constructed existence"],
      "displayPrimaryArgumentForm": null,
      "techniques": [
        {"id": "r4-prescribed-completion", "status": "provisional"}
      ]
    },
    "productionVerification": {
      "p1": "AI-led",
      "p2": "Hybrid informal-to-formal",
      "p3": "Compound pipeline",
      "p4": ["retrieval-augmented", "generator-verifier"]
    }
  },
  "problemHistory": {
    "problemId": "problem-anderson-quasi-completeness",
    "framingSource": null,
    "framingDate": null
  },
  "challengeContext": {
    "metricVersion": "1.0.0",
    "components": {
      "P": {"value": null, "evidenceState": "unknown"},
      "A": {"value": 2, "evidenceState": "observed"},
      "S": {"value": 1, "evidenceState": "observed"},
      "R": {"value": null, "evidenceState": "not_reported"}
    },
    "score": null,
    "category": "incomplete"
  },
  "metadata": {
    "systems": [], "artifacts": [], "humanMachineEvents": [],
    "verificationEvents": [],
    "derivedAssurance": {
      "ruleVersion": "1.0.0", "eventIds": [], "label": "reported only"
    },
    "coding": {}
  }
}
```

15 Worked example: Anderson's quasi-completeness problem

The tracked result is: there exists a Noetherian local ring that is weakly quasi-complete but not quasi-complete. The mathematical paper gives the result and a self-contained exposition, while the systems paper reports Rethlas discovery and Archon formalisation (Jiang et al., 2026; Ju et al., 2026).

Facet	Primary browse path and detail
Mathematics	13 Commutative algebra > 13J Topological rings and modules > 13J10 Complete rings, completion > Anderson quasi-completeness problem. Secondary setting: 13E05 Commutative Noetherian rings and modules.
Result and approach	Mathematical result > Negative resolution > Counterexample > Constructed existence. Specialist techniques: reduction via characterisations, prescribed completion, generic formal-fibre control and quotient/analytic-irreducibility obstruction.
Production	AI-led > Hybrid informal-to-formal > Compound multi-agent pipeline. Architecture features: theorem retrieval, generator–verifier loop, task decomposition, persistent memory and iterative compile–repair.
Verification	Natural-language model check; expert mathematical review; Lean kernel/build check; Comparator statement-correspondence and axiom check. Independent replication is not reported.
Challenge context	A=2 and S=1 are supported by the anchor claim as coded here. P, R and the framing date remain unknown pending a dedicated problem-history audit, so no total or category is derived in this example. This is intentional missingness, not a zero score.
Detail metadata	Rethlas, GPT-5.4 as the reported default model for its generation and verification agents, Matlas retrieval, Archon, LeanSearch, Lean 4, Mathlib and Comparator. Exact system facts remain metadata rather than navigation nodes.

The example illustrates why the result is the classification unit. The systems paper is deposited under computer-science categories, but the target result belongs in commutative algebra. It also illustrates why "formally verified" must expand into separate statement, kernel, axiom and human-review events. Comparator is designed precisely to separate a short statement specification from a large Lean development (Lean Community, 2025). Finally, it shows that a result can be fully classified in the three facets while its linked challenge context remains incomplete.

16 Governance and evolution

16.1 Versioning

The taxonomy follows semantic versioning in spirit:

- **Patch** changes correct labels, definitions or documentation without changing concept identity.
- **Minor** changes add concepts or optional fields compatibly.
- **Major** changes alter hierarchy, identifiers, counting rules or required fields incompatibly.

Every concept has a stable identifier, preferred label, definition, parent, status, introduction version and optional replacement. Labels may change without changing identity. Concepts are never deleted from historical data; they are deprecated and mapped.

Taxonomy, schema/profile, challenge metric, assurance-derivation rules and colour map have separate version identifiers. A record stores every applicable version. Changing a colour does not require a taxonomy release; changing the P/A/S/R formula does not silently recategorise a historical snapshot; and changing a display summary never deletes its contributing verification events. Published aggregates identify the data snapshot and all rule versions used to derive them.

16.2 Change operations

- `add`: introduce a new leaf or metadata field.
- `rename`: change the preferred display label while preserving the identifier.
- `broaden/narrow`: alter a concept definition with a documented migration.
- `split`: replace one concept with two or more narrower concepts and recode affected records.
- `merge`: replace several concepts with one broader concept while preserving historical mappings.
- `deprecate`: prevent new use while retaining the concept for old records.

These operations should be represented with SKOS-style broader, narrower, related and exact/close mapping relations where practical (Miles and Bechhofer, 2009; Arndt et al., 2021).

Public proposals for R4 or M4 include a proposed label, definition, parent or scope, evidence example, aliases and conflict check. A designated vocabulary reviewer assigns the identifier and status. Major interpretive disputes are recorded as open issues; they are not resolved by silently changing a result record.

16.3 Quality control

At minimum, every newly coded result should pass automated schema validation and a human check that:

- the primary MSC code describes the claim rather than the venue;
- the result path distinguishes settlement, partial progress and improvement;
- human and machine roles are stage-specific and evidence-backed;
- verification methods have explicit scope and independence;
- formal checks state their correspondence to the informal anchor claim;

- challenge components retain evidence and missingness, and any Superhuman override has second-coder review;
- unknown values have not been replaced by assumptions;
- source URLs, coding profile and applicable taxonomy/rule versions are present.

After the initial release gate, inter-coder agreement should be sampled periodically, especially for primary MSC, outcome, autonomy, challenge components and verification. Disagreements should improve definitions and examples rather than be resolved only by majority vote.

16.4 Phased adoption and deferred extensions

Implementation proceeds in four controlled phases:

1. **Baseline reconciliation:** align versions, identifiers, aliases, corpus counts and source snapshots; publish canonical registries and validation rules.
2. **Reliability pilot:** complete the exemplar set and inter-coder pilot; revise ambiguous definitions before freezing the v0.2 core.
3. **Core migration:** validate the existing pilot records first, then migrate the corpus under the Minimal profile and add Full enrichment where evidence permits.
4. **Explorer release:** expose progressive disclosure, evidence, profiles, conservative derivations, one-axis colour inheritance and the machine-readable colour map.

The following remain explicitly deferred until the core pilot is satisfactory: interactive selection among multiple colour axes; full M4 longitudinal event timelines; resource-class analysis; simultaneous-discovery and priority-dispute workflows; failure and near-miss records; public self-service vocabulary proposals; and semi-automated coding. The data model should not preclude these extensions, but v0.2 does not make them mandatory. Numerical importance scoring remains out of scope.

17 Implementation checklist for a future explorer

1. Store taxonomy concepts, R4/M4 registries and rule registries independently from result records.
2. Store ordered paths by stable concept identifier, not display text.
3. Preserve paper-level subject metadata separately from result-level assignments.
4. Support one primary and multiple secondary mathematical paths.
5. Validate Minimal and Full profiles with explicit unknown, not-reported and not-applicable states.
6. Support set-valued R3, optional display-primary, registered R4 techniques, multiple production features and complete verification events without flattening them into a single string.
7. Store challenge context outside the three facets and derive its score/category only from complete, versioned component evidence.

8. Compute assurance summaries conservatively from detailed data, retaining rule and event identifiers.
9. Make every parent node filter all descendants.
10. Retain taxonomy and record revision history.
11. Expose confidence and coding evidence in the interface.
12. Use the versioned colour registry, neutral roots, a single active structural axis and a separate status palette; reject arbitrary fallback colours.
13. Export a machine-readable snapshot with every published analysis.

References

Susanne Arndt, Patrick Ion, Mila Runnwerth, Moritz Schubotz, and Olaf Teschke. 10 years later: The mathematics subject classification and linked open data. In *Intelligent Computer Mathematics*, 2021. URL https://arxiv.org/abs/2107.13877.

Association for Computing Machinery. Artifact review and badging, 2020. URL https://www.acm.org/publications/policies/artifact-review-and-badging-current.

Won Ik Cho, Eunjung Cho, and Kyunghyun Cho. PaperCard for reporting machine assistance in academic writing. In *Equity and Access in Algorithms, Mechanisms, and Optimization*, 2023. URL https://arxiv.org/abs/2310.04824.

Walter Hernandez Cruz, Kamil Tylinski, Alastair Moore, Niall Roche, Nikhil Vadgama, Horst Treiblmaier, Jiangbo Shangguan, Paolo Tasca, and Jiahua Xu. Evolution of ESG-focused DLT research: An NLP analysis of the literature. *Quantitative Science Studies*, pages 1–30, 2025. doi: 10.1162/qss.a.7. URL https://arxiv.org/abs/2308.12420.

Jiedong Jiang, Yixiao Li, Zeming Sun, Yuefeng Wang, Liang Xiao, and Jiahong Yu. On some open problems in commutative algebra resolved by Rethlas, 2026. URL https://arxiv.org/abs/2605.25259.

Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, Zeming Sun, Shurui Liu, Leheng Chen, Yutong Wang, Yuefeng Wang, Zichen Wang, Wanyi He, Peihao Wu, Liang Xiao, Ruochuan Liu, Bryan Dai, and Bin Dong. Automated conjecture resolution with formal verification, 2026. URL https://arxiv.org/abs/2604.03789.

Lean Community. Comparator: A trustworthy judge for lean proof submissions, 2025. URL https://github.com/leanprover/comparator.

Lean FRO. Validating a lean proof, 2026. URL https://lean-lang.org/doc/reference/latest/ValidatingProofs/.

Mathematical Reviews and zbMATH. Mathematics subject classification 2020, 2020. URL https://msc2020.org/.

Alistair Miles and Sean Bechhofer. SKOS simple knowledge organization system reference. W3c recommendation, World Wide Web Consortium, 2009. URL https://www.w3.org/TR/skos-reference/.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.

National Information Standards Organization. CRediT: Contributor role taxonomy, 2022. URL https://credit.niso.org/contributor-roles-defined/.

Moritz Schubotz, Philipp Scharpf, Olaf Teschke, Andreas Kuehnemund, Corinna Breitinger, and Bela Gipp. AutoMSC: Automatic assignment of mathematics subject classification labels. In *Intelligent Computer Mathematics*, 2020. doi: 10.1007/978-3-030-53518-6_15.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024. doi: 10.1007/s11704-024-40231-1.

A Profile validation checklists

Minimal profile: publication gate

The smallest publishable result record contains all of the following, either as evidence-backed values or as an explicitly permitted missing state:

- stable result identifier, canonical anchor claim and at least one dated source with a declared role;
- taxonomy, profile, challenge-metric and applicable derivation-rule versions;
- one primary Mathematics path, or an explicit unclassified reason, with rationale and confidence;
- one R1–R2 path, all evidenced R3 values or `not_reported`, and an optional display-primary;
- P1 and P2 assignments with sources, evidence states and confidence;
- at least one verification event, or explicit `none_reported`, including target, method, outcome, scope and source;
- P/A/S/R component records with values or explicit missingness and no derived category when incomplete;
- coder, assignment date, classification rationale, overall confidence and validation status.

Full profile: enrichment gate

A record labelled Full additionally contains every applicable enrichment field and explicitly marks non-applicable ones:

- secondary MSC paths, external classifications and any M4 problem-history link;
- registered R4 techniques and detailed claim/outcome comparison;
- stage-specific human–machine events, P3/P4, exact system, harness, tool and resource metadata;
- complete verification-event provenance, independence, assumptions, artifacts and scope-correspondence assessment;
- full problem history and challenge evidence, including dated attempts, date precision, disputes and review;
- artifact, formal-trust and reproducibility metadata; and field-level review, adjudication and migration history.

`unknown` means the value cannot currently be established; `not_reported` means the reviewed sources omit it; `not_applicable` means the field does not conceptually apply. These states are not interchangeable.

B Version history

Version 0.2.0

Published 4 August 2026. Introduces formal Minimal and Full coding profiles; the separate, versioned challenge-context and problem-history layer; an auditable P/A/S/R metric retaining the Superhuman category; explicit missingness; optional R3 display-primary; provisional/stable R4 governance; statement-correspondence scope; conservative assurance derivation; exemplar and decision-guide requirements; an inter-coder release gate; and phased implementation. Versions the colour map, activates one structural axis at a time, separates status styling and forbids arbitrary colour fallbacks.

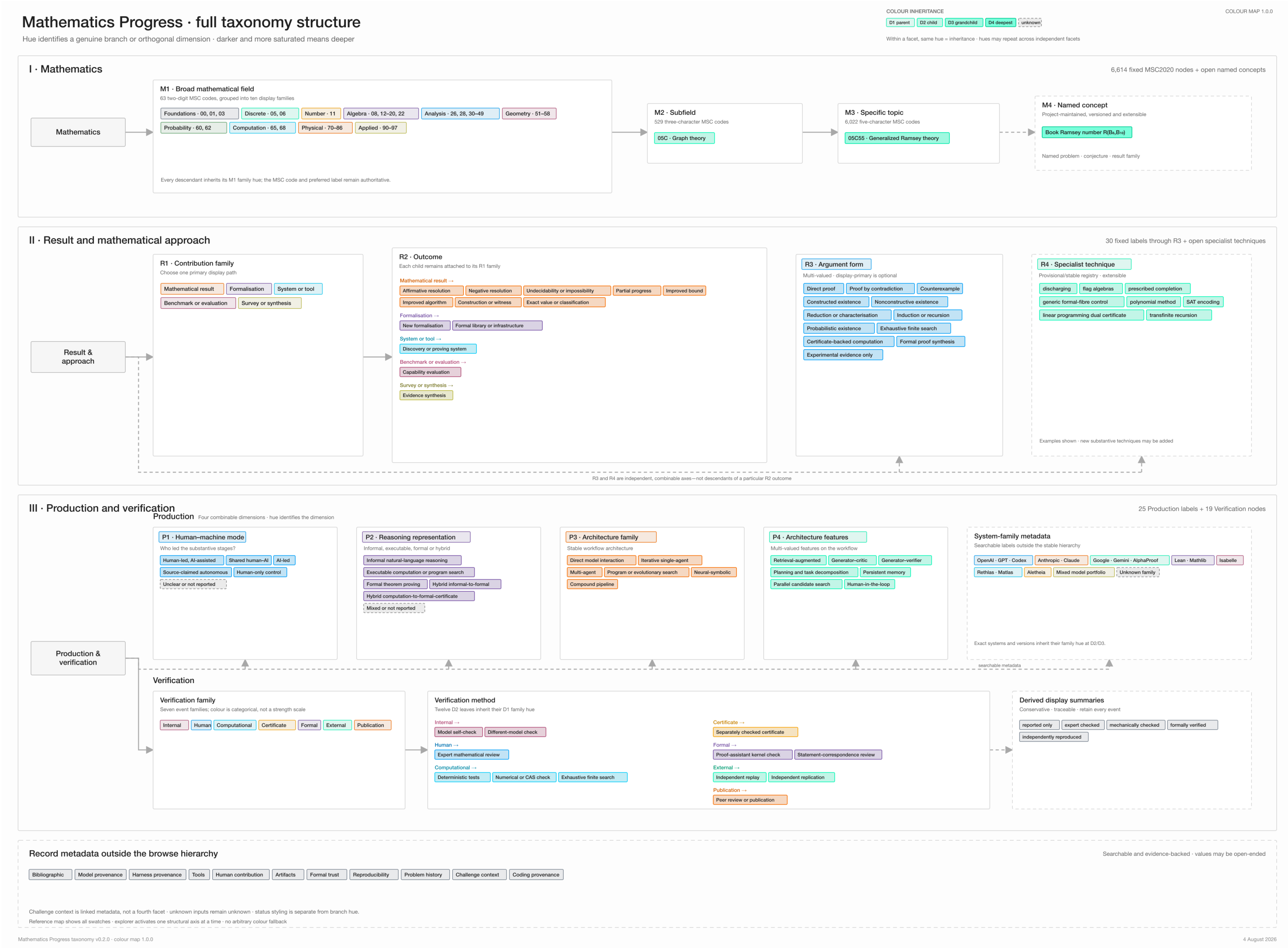
Version 0.1.1

Published 4 August 2026. Adds the explorer colour-inheritance grammar, removes arbitrary fallback colouring from the proposed scheme, distinguishes hierarchical inheritance from orthogonal axes and includes the full colour map as an appendix. No concept identifiers or classification rules change.

Version 0.1.0

Published 3 August 2026. First operational hierarchy. Establishes result-level classification, the three top-level browse facets, MSC2020 inheritance, stage-specific production metadata, event-based verification and progressive-disclosure requirements.

C Full taxonomy structure and colour-inheritance map



Reading note. This reference map displays all registered axis swatches together; the explorer activates only one structural colour axis at a time. Within an axis, hue identifies a branch or dimension and darker, more saturated labels indicate greater depth. Repeated hues in independent facets do not imply a relationship. Roots remain neutral, status uses separate styling, solid connectors show inheritance, dashed connectors show combinable axes or metadata, and dashed containers show extensible vocabulary.